

Component 1:

Problem Analysis & AI System Design

Problem Analysis

The problem is to find the shortest route between 2 or 3 locations on the Loughborough University campus, which is represented as a weighted graph where the weight is the distance in metres. The agent starts at a specified building and must reach the goal building while minimising the total distance. I have 3 real world constraints which are introduced as twists. The first is that construction works nearby EHB temporarily block the direct path between EHB and James France. Second is that the path between James France and The Holt closes at night, forcing the agent to find a different path when it is night time. The set of blocked roads is determined before the search begins. The third is that sometimes, people like to pass other buildings during their journey, such as going past the Student Union to buy something before reaching the destination.

The environment can be characterised as fully observable, the agent has complete knowledge of all the buildings, paths, distances and which roads are blocked before it begins searching. It is deterministic since travelling a given path always results in the expected movement at the expected cost. It is static as the map does not change during execution. It is discrete as there is a finite set of buildings and connections.

Justification of Approach

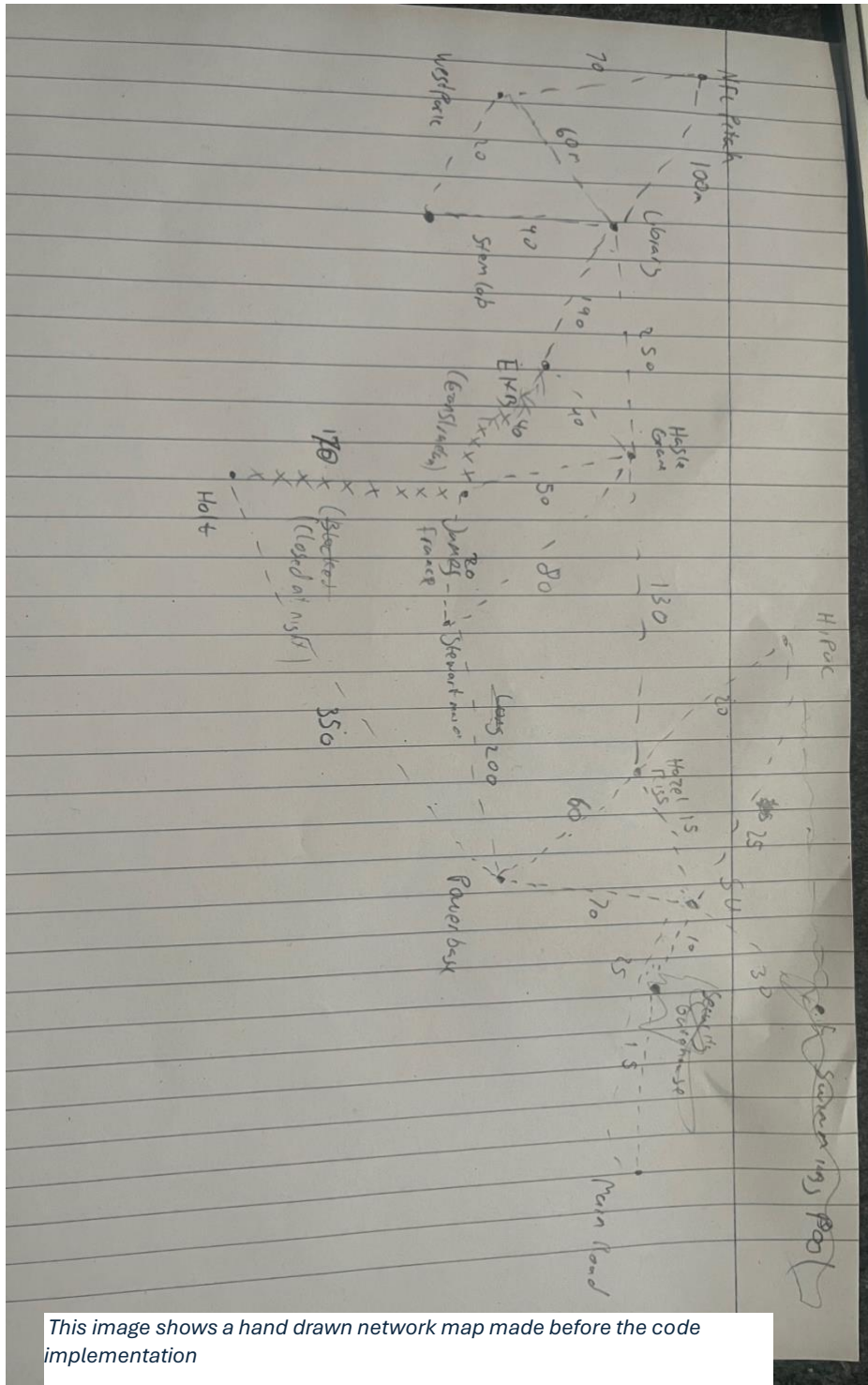
I chose A* search as the AI approach because it is an algorithm that combines the actual cost from the start node $g(n)$ with the heuristic estimate of the remaining distance to the goal $h(n)$. Nodes with the lowest f score ($f(n) = g(n) + h(n)$) are expanded first.

A reason why A* is the most suitable is because it is complete, it will always find a route if one exists, which matters when roads are blocked and the agent must reason around them. Another reason is because it is optimal. It is guaranteed to return the lowest cost path. Straight line distance between node coordinates is used as the heuristic. Thirdly, it is efficient, the heuristic guides the search toward the goal, reducing unnecessary node expansions compared to uninformed methods.

Breadth First Search is a less appropriate alternative as it treats all edges as equal and expands nodes purely by number of hops, ignoring weights. This means that BFS wouldn't find the shortest path but instead the path with the least nodes visited, which could be longer depending on the weights. This is the wrong objective. A greedy Best first search algorithm would be better than BFS as it uses path costs, however it still doesn't take heuristics into account, making it less efficient than A*.

System Representation

The campus is represented as a weighted graph stored as a nested dictionary containing all nodes its linked to with the weight. Blocked roads will be stored in a list containing pairs of nodes. Each building also has its on (x,y) coordinates which is used to visually represent the map and calculate the heuristic. The Agent is a goal based agent which uses a start node, waypoint node if needed, a goal node and the current time of day.



Algorithmic Plan

Function `ASTAR_SEARCH(initial state, goal state )`:

`node state` $\leftarrow$ initial state

`node path` $\leftarrow$ [Initial state] //initialised with the first node in the list

`g` $\leftarrow$ 0 //path cost of the node, 0 at the start

`f` $\leftarrow$ `g + heuristic()` //gets the heuristic and adds it to g

if `isGoal(node state)`: return node path

`frontier` $\leftarrow$ priority queue, insert f, g, node state, node path

`explored` $\leftarrow$ empty set

loop:

 if frontier is empty: return failure

 f, g, node state, node path $\leftarrow$ POP lowest f from frontier

 if `isGoal(node state)`: return node_path

 add node state to explored

 for each action in `ACTIONS(node state)`:

 child state $\leftarrow$ action

 if(`node state, child state`) in blocked roads: skip

 child_g $\leftarrow$ g + cost to child

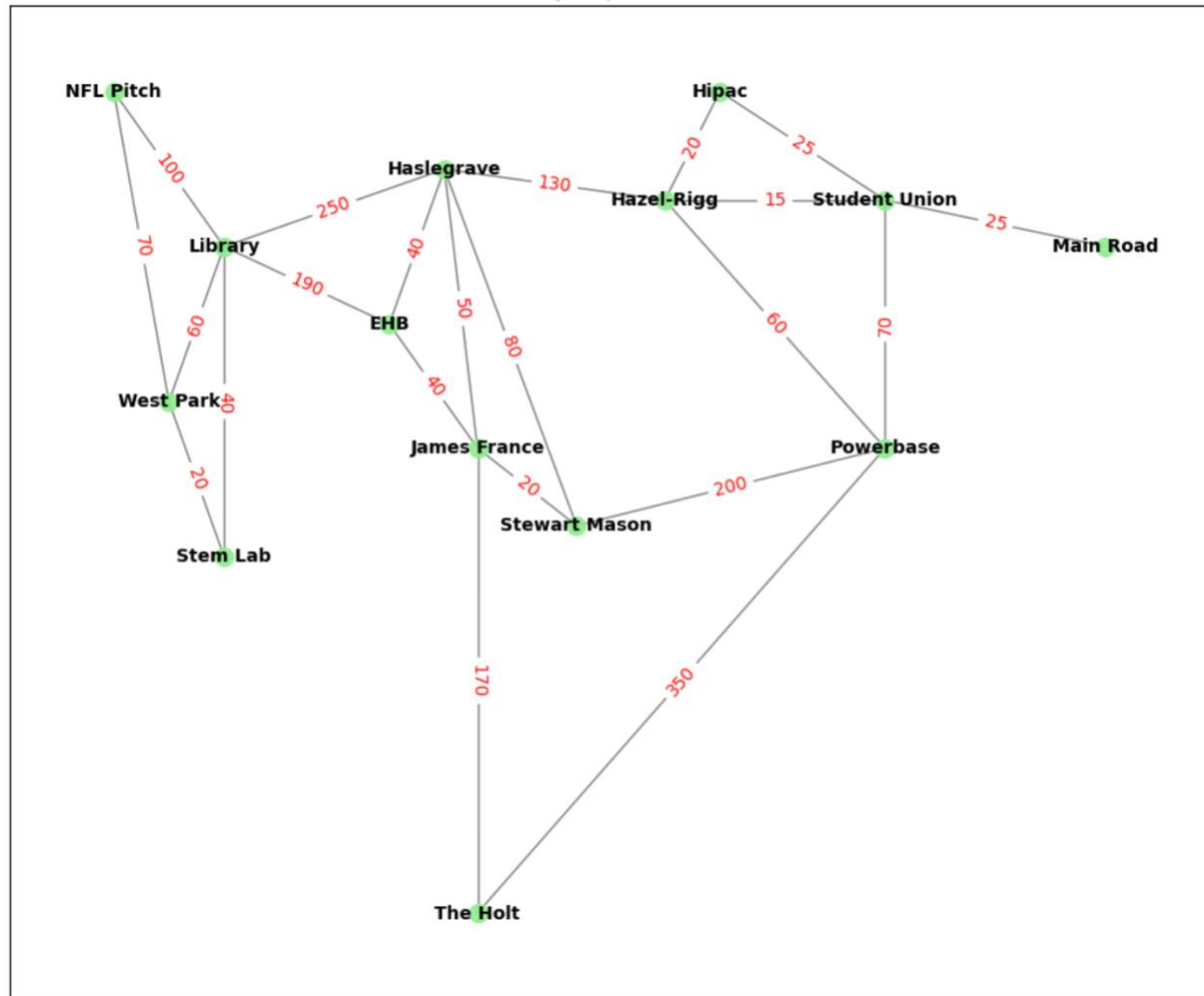
 child f $\leftarrow$ child g + heuristic()

 if child state not in explored and not in frontier:

 Insert child f, child g, child state, path + [child state] into frontier

Another A* search function which will also take in the waypoint node, then do 2 normal A star searches and combining the paths

Map Made in the System using matplotlib and network



LINK: <https://colab.research.google.com/drive/112vEY3yLDF-vG5KFR49YqM7BMTSqeN7E?usp=sharing>

Component 3:

Reflection and Ethical Analysis

Reflection

The A* system successfully finds the optimal route across a graph representation of Loughborough University's campus. Every decision is fully traceable, the frontier is printed at each step, showing the exact f, g and h values for each expansion. This allows someone to follow the search step by step and verify that the path chosen is correct. The System is also deterministic, given the same start, goal and time of day will always produce the same output.

The blocked road mechanism demonstrates meaningful reasoning too. During the day, the EHB construction closure forces that the agent to route via Haslegrave rather than taking the direct route to

James France. At night, the James France – The Holt closure changes the optimal route further, showing the system adapting to different real world conditions without any randomness.

However the system suffers from limitations. The graph is simplified a lot, real paths curve, vary in width or some paths can be congested during the day. Distances are fixed estimates. The map is also static within search meaning that it cannot respond to a path becoming blocked mid journey. The coordinate based heuristic doesn't use real metres, meaning that $h(n)$ and $g(n)$ are not numerically comparable.

Contrasts with a Deep Learning Route Planner

A modern deep learning approach to this problem would resemble systems like google maps, which use neural networks trained on vast quantities of real world data, for example, traffic patterns, live sensor feeds and so on. Rather than explicitly representing the campus as a graph with fixed edge weights, such a system would learn implicit representations of routes from data, which helps to catch patterns humans might not anticipate.

In terms of capability and efficiency, the deep learning system would outperform the A* implementation by a whole lot. It could handle maps with thousands of nodes, dynamically re route around new obstacles and personalise routes based on individual preference or mobility needs, eg bus routes, walking routes. The A* system would require more manual definitions of every node, edge and costs and would need to be rebuilt if the campus changed significantly.

However in terms of transparency, the A* system is far superior. Every step of the search can be expected and explained using the f score. A deep learning systems routing decisions emerge from millions of parameters with no interpretable logic that a human can read. If it makes a wrong recommendation, it is very difficult to determine why.

Ethical risks in the A* System

The A* system itself carries relatively limited ethical risk, mainly due to its simplicity and transparency. Errors can be identified and corrected straightforwardly. However several risks exist, such as the graph being manually constructed means it has to be accurate, any mistakes in edge weights or missing connections are the designers fault. There is also no consideration of accessibility, the system treats all all paths as equally navigable, which could (hypothetically) direct a wheelchair user down steps. In addition to this, it doesn't take safety into account, it could recommend short routes that pass through slightly dangerous short cuts which is needed to be walked through during the day and not in the night where it is poorly lit (hypothetically again).

Ethical risks in a Deep Learning alternative

A deep learning route planner amplifies several of these risks and also introduces new ones. The most significant concern is data privacy. Training such a system on real GPS traces requires collecting detailed records of where people go and when, which raises concern about consent and data retention. If the data isn't treated securely, information could be leaked putting people who relate to the data at

high risk Another concern is bias and fairness. If the training data over represents certain types of users, the system may produce suboptimal or even harmful routes for other users, eg users with disabilities, heavy luggage vs a athletic able bodied student walking quickly. Another risk is over reliance and accountability. Users of a deep learning navigation system may follow its recommendations without question. If the system fails due to a something such as a data error, the consequence could range from inconvenience to genuine harm. For example, the system could lead someone to a place, fail then cause them to be stranded/lost for a while. Finally, there is the risk of scalability of harm. A bug in the A* system would only affect one user running one notebook. A bug in a deep learning system used by thousands or millions of people simultaneously could misdirect large numbers of people at once.

Comparison

The comparison between my A* and a deep learning alternative illustrates a fundamental trade off in AI system design: prioritising capability or explainability. The deep learning system is more powerful and more scalable but it is harder to correct, more dependent on potentially sensitive data and more likely to produce failures that are difficult to explain. For a campus navigation problem, increased technical power does not automatically lead to a better or more ethical outcome. A system that a student can read, understand and correct is believed to be more appropriate for this. This could suggest that the choice of AI method should always be driven by the requirements of the context, not just by performance metrics